

Lab 6: Matching, Weighting, and Doubly Robust Estimation

SOCIOL 723, Social Statistics II, Thursday, October 1

Wenhao Jiang

Table of contents

1 The question

Estimand. The average difference in log personal earnings between completing a bachelor’s degree and not completing one, among the full-time workers aged 25–64 in our GSS analytic sample, 2010–2022.

Note

Note the deliberately modest population. The GSS is a complex sample, and doing this properly for the U.S. population would require the survey weights (`wt`) and PSU/stratum-aware inference. We work unweighted here so that the adjustment methods are visible on their own; that is a simplification, and you should say so if you write it up.

We also exclude `hours` from the adjustment set. Hours worked is plausibly a *consequence* of completing college, so conditioning on it would be the post-treatment error from Week 5; it would answer a different question.

This is an *observational* question with obvious confounding: people who complete college differ from those who do not in family background, measured ability, and much else. We are going to assume conditional ignorability given the covariates we have, do the adjustment five different ways, and then ask how much an unobserved confounder would have to matter to overturn the answer.

Warning

Assuming conditional ignorability here is almost certainly wrong. That is the point of the last section. Treat today’s numbers as an exercise in *method*, not as an estimate of the return to college.

```

library(dplyr)
library(ggplot2)
library(MatchIt)
library(cobalt)
library(sensemakr)
library(sandwich); library(lmtest)
library(boot)

gss <- readRDS("../Data/gss_earnings.rds") |>
  mutate(D = ba, Y = llearn)

covs <- c("pareduc", "wordsum", "female", "black", "otherace", "exper")
c(n = nrow(gss), treated = sum(gss$D), control = sum(1 - gss$D))
#>      n treated control
#>   3509    1441    2068

```

2 Where we start: the naive comparison

```

naive <- lm(Y ~ D, data = gss)
reg <- lm(Y ~ D + pareduc + wordsum + female + black + otherace + exper +
  I(exper^2), data = gss)

c(naive = coef(naive)["D"], regression = coef(reg)["D"])
#>      naive.D regression.D
#>      0.6834      0.6295

```

Adjustment moves the estimate substantially. But regression is only one of the adjustment strategies available, and it makes assumptions we have not examined.

```

bal.tab(reg_formula <- D ~ pareduc + wordsum + female + black + otherace +
  exper, data = gss, un = TRUE, thresholds = c(m = 0.1))
#> Balance Measures
#>      Type Diff.Un      M.Threshold.Un
#> pareduc Contin.  0.736 Not Balanced, >0.1
#> wordsum Contin.  0.793 Not Balanced, >0.1
#> female  Binary  0.055  Balanced, <0.1
#> black   Binary -0.053  Balanced, <0.1
#> otherace Binary -0.016  Balanced, <0.1
#> exper   Contin. -0.420 Not Balanced, >0.1
#>
#> Balance tally for mean differences
#>      count

```

```

#> Balanced, <0.1      3
#> Not Balanced, >0.1  3
#>
#> Variable with the greatest mean difference
#> Variable Diff.Un      M.Threshold.Un
#> wordsum    0.793 Not Balanced, >0.1
#>
#> Sample sizes
#>      Control Treated
#> All      2068      1441

```

The `Diff.Un` column is the standardized mean difference. Values well above 0.1, here `pareduc` and `wordsum` especially, mean the two groups are not comparable as they stand.

3 Exact matching on cells

The lecture's matching section began with the cleanest estimator there is: coarsen the covariates into cells, take the difference in means inside every cell that holds both graduates and non-graduates, and average the cell differences with the weights of the estimand you named. This section reproduces the lecture's numbers by hand, then checks them against `MatchIt`.

3.1 Building the cells

```

## coarsen the three continuous covariates at rough terciles; keep the binaries
as they are
gss <- gss |>
  mutate(pareduc_b = cut(pareduc, c(-Inf, 11, 13, Inf), labels = c("<12",
    "12-13", "14+")),
    wordsum_b = cut(wordsum, c(-Inf, 5, 7, Inf), labels = c("0-5",
    "6-7", "8-10")),
    exper_b = cut(exper, c(-Inf, 16, 29, Inf), labels = c("0-16",
    "17-29", "30+")),
    cell = interaction(female, black, pareduc_b, wordsum_b, exper_b))
## one row per cell: how many treated and untreated, and their mean outcomes
cells <- gss |>
  group_by(cell) |>
  summarise(n1 = sum(D), n0 = sum(1 - D),
    m1 = mean(Y[D == 1]), m0 = mean(Y[D == 0]), .groups = "drop") |>
  mutate(n = n1 + n0, tau = m1 - m0, both = n1 > 0 & n0 > 0)
c(possible = nlevels(gss$cell), occupied = nrow(cells), both_arms =
sum(cells$both),
  treated_only = sum(cells$n1 > 0 & cells$n0 == 0), untreated_only =
sum(cells$n0 > 0 & cells$n1 == 0))

```

```

#>      possible      occupied      both_arms      treated_only      untreated_only
#>          108          106          92          2          12
## coverage: who is lost when the one-armed cells are dropped
c(treated_dropped = sum(cells$n1[!cells$both]), untreated_dropped =
sum(cells$n0[!cells$both]))
#>      treated_dropped      untreated_dropped
#>           4           70

```

3.2 The three estimands, and what OLS returns

```

m <- filter(cells, both)
## ATE: cell shares among matched units; ATT: shares among matched treated;
ATU: among matched untreated
c(naive = mean(gss$Y[gss$D == 1]) - mean(gss$Y[gss$D == 0]),
  ATE = sum(m$tau * m$n) / sum(m$n),
  ATT = sum(m$tau * m$n1) / sum(m$n1),
  ATU = sum(m$tau * m$n0) / sum(m$n0),
  ## OLS saturated in the cells: the variance-weighted average from the
  lecture
  OLS_cells = coef(lm(Y ~ D + cell, data = gss))["D"])
#>      naive      ATE      ATT      ATU      OLS_cells.D
#>      0.6834      0.6346      0.6247      0.6417      0.6305

```

The four adjusted numbers are the same 92 cell differences under four sets of weights. Bootstrap the whole procedure for standard errors; resampling units is valid here because nothing is nearest-neighbour.

```

boot_exact <- function(d) {
  b <- d[sample(nrow(d), replace = TRUE), ]
  t <- b |> group_by(cell) |>
    summarise(n1 = sum(D), n0 = sum(1 - D), y1 = sum(Y * D), y0 = sum(Y * (1 -
      D)), .groups = "drop") |>
    filter(n1 > 0, n0 > 0) |> mutate(tau = y1 / n1 - y0 / n0)
  c(ATE = sum(t$tau * (t$n1 + t$n0)) / sum(t$n1 + t$n0), ATT = sum(t$tau *
    t$n1) / sum(t$n1))
}
set.seed(723)
apply(replicate(200, boot_exact(gss)), 1, sd)
#>      ATE      ATT
#> 0.03295 0.03995

```

3.3 The checks: coverage and balance

```
## standardized mean difference: (weighted mean treated - weighted mean
untreated) / pooled SD
covs <- c("pareduc", "wordsum", "exper", "female", "black", "otherace")
sd_pool <- sapply(covs, function(v) sqrt((var(gss[gss$D == 1, v]) +
var(gss[gss$D == 0, v])) / 2))
smd <- function(w1, w0) sapply(covs, function(v)
  (weighted.mean(gss[[v]], w1) - weighted.mean(gss[[v]], w0)) / sd_pool[v])
## weights that reproduce the matched estimators: each unit gets (target share
of its cell) / (units of its arm in the cell)
cw <- as.character(gss$cell)
share <- setNames(m$n / sum(m$n), as.character(m$cell))      # ATE target:
matched-population cell shares
share1 <- setNames(m$n1 / sum(m$n1), as.character(m$cell))   # ATT target:
treated cell shares
n1c <- setNames(m$n1, as.character(m$cell)); n0c <- setNames(m$n0,
as.character(m$cell))
get <- function(v, k) ifelse(is.na(v[k]), 0, v[k])
w1_ate <- gss$D * get(share, cw) / pmax(get(n1c, cw), 1)
w0_ate <- (1 - gss$D) * get(share, cw) / pmax(get(n0c, cw), 1)
w1_att <- gss$D * get(share1, cw) / pmax(get(n1c, cw), 1)
w0_att <- (1 - gss$D) * get(share1, cw) / pmax(get(n0c, cw), 1)
round(rbind(raw = smd(gss$D, 1 - gss$D), ATE_matched = smd(w1_ate, w0_ate),
ATT_matched = smd(w1_att, w0_att)), 3)
#>      pareduc.pareduc wordsum.wordsum exper.exper female.female
#> raw      0.736      0.793      -0.420      0.111
#> ATE_matched 0.125      0.123      -0.108      0.000
#> ATT_matched 0.141      0.109      -0.093      0.000
#>      black.black otherace.otherace
#> raw      -0.15      -0.052
#> ATE_matched 0.00      0.070
#> ATT_matched 0.00      0.095
```

Exactly matched variables balance by construction; the coarsened ones keep the within-band imbalance the bands allow; `otherace`, which we did not match on, moves on its own. None of these numbers is a test.

3.4 The same thing in MatchIt

```
## method = "exact" builds the same cells; estimand chooses the weights
for (est in c("ATE", "ATT")) {
  mm <- matchit(D ~ female + black + pareduc_b + wordsum_b + exper_b,
```

```

        data = gss, method = "exact", estimand = est)
md <- match.data(mm)
cat(est, ":", round(coef(lm(Y ~ D, data = md, weights = weights))["D"], 3),
    " matched n =", nrow(md), "\n")
}
#> ATE : 0.635  matched n = 3435
#> ATT : 0.625  matched n = 3435

```

4 The propensity score

```

ps_mod <- glm(D ~ pareduc + wordsum + female + black + otherace +
              exper + I(exper^2),
              data = gss, family = binomial)
gss$ps <- fitted(ps_mod)

summary(gss$ps)
#>      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
#> 0.00265 0.21792 0.39107 0.41066 0.59004 0.94567

```

```

ggplot(gss, aes(ps, fill = factor(D))) +
  geom_density(alpha = .45, colour = NA) +
  scale_fill_manual(values = c("0" = "navy", "1" = "firebrick"),
                    labels = c("no BA", "BA"), name = NULL) +
  labs(x = "estimated propensity score", y = "density") +
  theme_minimal(base_size = 10)

```

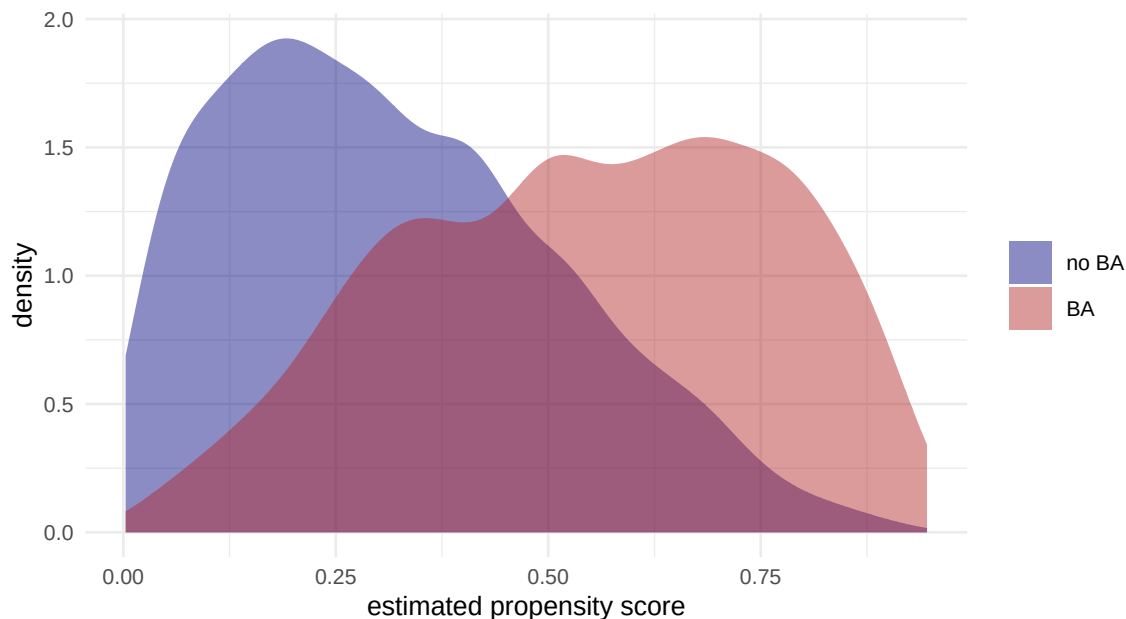


Figure 1: Distribution of the estimated propensity score by treatment status. Overlap is imperfect in both tails.

```
## how many treated units sit where controls are scarce, and vice versa?
c(treated_above_0.9 = sum(gss$D == 1 & gss$ps > 0.9),
  controls_above_0.9 = sum(gss$D == 0 & gss$ps > 0.9),
  controls_below_0.1 = sum(gss$D == 0 & gss$ps < 0.1),
  treated_below_0.1 = sum(gss$D == 1 & gss$ps < 0.1))
#> treated_above_0.9 controls_above_0.9 controls_below_0.1 treated_below_0.1
#>                34                3                305                28
```

i Note

Note the trade-off from lecture: a propensity model that predicts treatment *well* is a model that separates the groups, which is exactly the situation in which comparison is hardest. Judge the model by **balance**, not by AUC.

5 Matching

5.1 Nearest-neighbour on the propensity score

```
m_nn <- matchit(D ~ pareduc + wordsum + female + black + otherace +
  exper + I(exper^2),
  data = gss, method = "nearest", distance = "glm",
  estimand = "ATT", replace = FALSE)
m_nn
```

```
#> A `matchit` object
#> - method: 1:1 nearest neighbor matching without replacement
#> - distance: Propensity score
#> - estimated with logistic regression
#> - number of obs.: 3509 (original), 2882 (matched)
#> - target estimand: ATT
#> - covariates: pareduc, wordsum, female, black, otherace, exper, I(exper^2)
```

```
love.plot(m_nn, thresholds = c(m = 0.1), binary = "std",
          colors = c("firebrick", "navy")) +
  theme_minimal(base_size = 9)
```

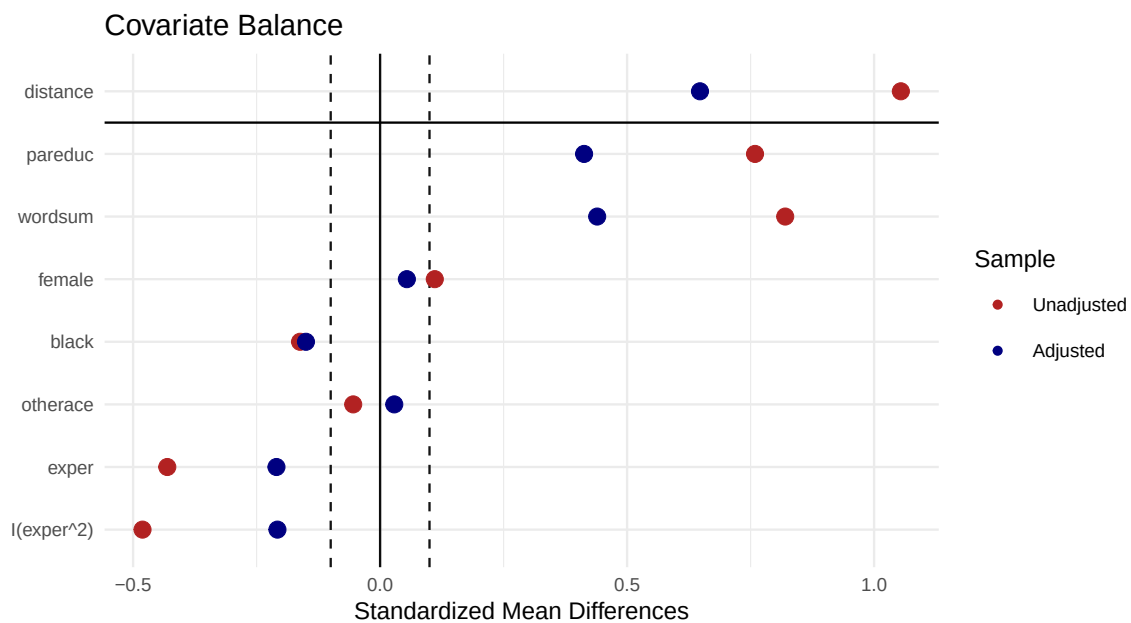


Figure 2: Covariate balance before and after nearest-neighbour matching. Everything should land inside the dashed lines.

```
md_nn <- match.data(m_nn)
fit_nn <- lm(Y ~ D + pareduc + wordsum + female + black + otherace +
             exper + I(exper^2),
             data = md_nn, weights = weights)
round(coefestest(fit_nn, vcov = vcovCL, cluster = ~ subclass)["D", ], 4)
#> Estimate Std. Error t value Pr(>|t|)
#> 0.6260 0.0319 19.6506 0.0000
```

Two things to notice. First, we still run a regression on the matched data, matching is *preprocessing*, and the regression mops up residual imbalance. Second, the standard errors are clustered on the matched pair, because the matching induces dependence.

5.2 Mahalanobis distance

Instead of collapsing the covariates to a propensity score, match on the covariates directly, scaling by their covariance so that no variable dominates through its units.

```
m_mah <- matchit(D ~ pareduc + wordsum + female + black + otherace +
  exper,
  data = gss, method = "nearest", distance = "mahalanobis",
  estimand = "ATT")
summary(m_mah)$nn
```

#>	Control	Treated
#> All (ESS)	2068	1441
#> All	2068	1441
#> Matched (ESS)	1441	1441
#> Matched	1441	1441
#> Unmatched	627	0
#> Discarded	0	0

5.3 A caliper

A caliper refuses matches beyond a set distance on the propensity score, measured in standard deviations of the linear predictor.

```
m_cal <- matchit(D ~ pareduc + wordsum + female + black + otherace +
  exper + I(exper^2),
  data = gss, method = "nearest", distance = "glm",
  estimand = "ATT", caliper = 0.1, std.caliper = TRUE)
summary(m_cal)$nn
```

#>	Control	Treated
#> All (ESS)	2068	1441
#> All	2068	1441
#> Matched (ESS)	1017	1017
#> Matched	1017	1017
#> Unmatched	1051	424
#> Discarded	0	0

Look at the **Unmatched** row for the treated. Those respondents have no comparable counterpart in the data, and dropping them silently redefines the estimand; you are now describing the subpopulation of degree-holders who have plausible non-degree counterparts.

5.4 Coarsened exact matching

```
m_cem <- matchit(D ~ pareduc + wordsum + female + black + exper,
  data = gss, method = "cem", estimand = "ATT")
```

```
summary(m_cem)$nn
#>           Control Treated
#> All (ESS)      2068.0    1441
#> All           2068.0    1441
#> Matched (ESS)   521.1     843
#> Matched        1119.0     843
#> Unmatched       949.0     598
#> Discarded        0.0        0
```

CEM is the most transparent method: it matches exactly within coarsened bins, so balance is guaranteed by construction. The price is written plainly in the number of discarded observations.

6 Weighting

6.1 IPW by hand

```
e <- gss$ps
D <- gss$D
Y <- gss$Y

## Horvitz-Thompson
ate_ht <- mean(D * Y / e) - mean((1 - D) * Y / (1 - e))

## Hajek / stabilized -- divide by the sum of the weights
w1 <- D / e
w0 <- (1 - D) / (1 - e)
ate_sipw <- sum(w1 * Y) / sum(w1) - sum(w0 * Y) / sum(w0)

c(horvitz_thompson = ate_ht, stabilized = ate_sipw)
#> horvitz_thompson      stabilized
#>           1.5371           0.6023
```

```
w <- ifelse(D == 1, 1 / e, 1 / (1 - e))
ess <- sum(w)^2 / sum(w^2)
c(max_weight = max(w), ess = ess, n = length(w), ess_share = ess / length(w))
#> max_weight      ess      n  ess_share
#>   105.4685  1241.7073 3509.0000    0.3539
```

The **effective sample size** is what your standard errors should reflect. If a handful of observations carry most of the weight, you have far less information than n suggests.

6.2 ATT and overlap weights

```
att_w <- ifelse(D == 1, 1, e / (1 - e))      # ATT weights
ato_w <- ifelse(D == 1, 1 - e, e)            # overlap weights

wmean_diff <- function(wt) {
  sum(wt * Y * D) / sum(wt * D) - sum(wt * Y * (1 - D)) / sum(wt * (1 - D))
}
c(ATE_stabilized = ate_sipw,
  ATT = wmean_diff(att_w),
  ATO_overlap = wmean_diff(ato_w))
#> ATE_stabilized      ATT      ATO_overlap
#>      0.6023      0.6310      0.6371
```

Three different numbers, because they are three different **estimands**, not three attempts at one. The overlap-weighted estimate describes the subpopulation where comparison is most informative, and its weights are bounded by construction.

6.3 Does weighting actually balance the covariates?

```
bal.tab(D ~ pareduc + wordsum + female + black + otherace + exper,
  data = gss, weights = ifelse(D == 1, 1 / e, 1 / (1 - e)),
  method = "weighting", un = TRUE, thresholds = c(m = 0.1))
#> Balance Measures
#>      Type Diff.Un Diff.Adj      M.Threshold
#> pareduc  Contin.  0.736  -0.218 Not Balanced, >0.1
#> wordsum  Contin.  0.793  -0.077  Balanced, <0.1
#> female   Binary  0.055  -0.027  Balanced, <0.1
#> black    Binary -0.053   0.026  Balanced, <0.1
#> otherace Binary -0.016   0.021  Balanced, <0.1
#> exper    Contin. -0.420   0.006  Balanced, <0.1
#>
#> Balance tally for mean differences
#>      count
#> Balanced, <0.1      5
#> Not Balanced, >0.1  1
#>
#> Variable with the greatest mean difference
#> Variable Diff.Adj      M.Threshold
#> pareduc  -0.218 Not Balanced, >0.1
#>
#> Effective sample sizes
#>      Control Treated
```

```
#> Unadjusted      2068  1441.
#> Adjusted        1623   410.4
```

7 G-computation and AIPW

```
## fit separate outcome models in each arm, then impute both potential
outcomes
out_form <- Y ~ pareduc + wordsum + female + black + otherace + exper +
  I(exper^2)
m1 <- lm(out_form, data = gss[D == 1, ])
m0 <- lm(out_form, data = gss[D == 0, ])

mu1 <- predict(m1, newdata = gss)
mu0 <- predict(m0, newdata = gss)

ate_gcomp <- mean(mu1 - mu0)
ate_gcomp
#> [1] 0.6318
```

```
aipw_terms <- (mu1 - mu0) +
  D * (Y - mu1) / e -
  (1 - D) * (Y - mu0) / (1 - e)

ate_aipw <- mean(aipw_terms)

## the influence-function standard error comes free
se_aipw <- sd(aipw_terms) / sqrt(length(aipw_terms))

c(estimate = ate_aipw, se = se_aipw,
  lower = ate_aipw - 1.96 * se_aipw, upper = ate_aipw + 1.96 * se_aipw)
#> estimate      se      lower      upper
#> 0.60512 0.03981 0.52709 0.68315
```

Tip

The AIPW estimator is an average of n i.i.d. terms, so its standard error is just the standard deviation of those terms over $\sqrt{n}$. That is not a coincidence, `aipw_terms` is the *efficient influence function*, and this is the pattern that Week 13 generalizes.

7.1 Everything in one place

```
results <- tibble::tibble(
  method = c("Naive difference", "OLS adjustment", "NN matching (ATT)",
             "IPW stabilized (ATE)", "ATT weights", "Overlap weights (ATO)",
             "G-computation (ATE)", "AIPW (ATE)"),
  estimate = c(coef(naive)["D"], coef(reg)["D"], coef(fit_nn)["D"],
               ate_sipw, wmean_diff(att_w), wmean_diff(ato_w),
               ate_gcomp, ate_aipw)
)
knitr::kable(results, digits = 4)
```

method	estimate
Naive difference	0.6834
OLS adjustment	0.6295
NN matching (ATT)	0.6260
IPW stabilized (ATE)	0.6023
ATT weights	0.6310
Overlap weights (ATO)	0.6371
G-computation (ATE)	0.6318
AIPW (ATE)	0.6051

If these had disagreed sharply, that itself would be the finding: it would tell us the answer is coming from modelling choices rather than from comparison.

7.2 Bootstrapping the whole procedure

Standard errors must account for the fact that the propensity score was *estimated*. The safest route is to bootstrap the entire pipeline.

```
aipw_fun <- function(data, idx) {
  d <- data[idx, ]
  psm <- glm(D ~ pareduc + wordsum + female + black + otherace +
             exper + I(exper^2), data = d, family = binomial)
  ee <- fitted(psm)
  f1 <- lm(out_form, data = d[d$D == 1, ]); f0 <- lm(out_form, data = d[d$D ==
0, ])
  m1p <- predict(f1, newdata = d); m0p <- predict(f0, newdata = d)
  mean((m1p - m0p) + d$D * (d$Y - m1p) / ee -
       (1 - d$D) * (d$Y - m0p) / (1 - ee))
}
set.seed(723)
bo <- boot(gss, aipw_fun, R = 500)
```

```
c(estimate = bo$t0, boot_se = sd(bo$t), analytic_se = se_aipw)
#>      estimate      boot_se analytic_se
#>      0.60512      0.03579      0.03981
```

The two standard errors agree closely, which is reassuring: the influence-function formula is doing its job.

8 Sensitivity analysis

All of the above assumed we measured every confounder. We did not. So ask instead: **how strong would an unobserved confounder have to be?** Two answers, both computed from the long regression `reg` and both on the slides.

8.1 Oster's coefficient-stability bound

Oster (2019) compares the coefficient *without* controls (short regression) to the coefficient *with* them (long regression), and scales the movement by how much R^2 the controls added. If controls moved the coefficient little but also explained little, that is no comfort.

```
## the short regression: Y on D alone; the long one is `reg` from above
short <- lm(Y ~ D, data = gss)
b_dot <- unname(coef(short)["D"]); r_dot <- summary(short)$r.squared ##
uncontrolled
b_til <- unname(coef(reg)["D"]); r_til <- summary(reg)$r.squared ##
controlled
c(beta_short = b_dot, r2_short = r_dot, beta_long = b_til, r2_long = r_til)
#> beta_short  r2_short  beta_long  r2_long
#>      0.6834      0.1295      0.6295      0.2434

## Oster's assumption: selection on unobservables = delta x selection on
observables,
## and Rmax = the R^2 we would reach with the unobservables in.
## The movement per unit of R^2 gained is extrapolated to the R^2 still to
gain:
oster_beta <- function(delta, rmax) b_til - delta * (b_dot - b_til) * (rmax -
r_til) / (r_til - r_dot)
## delta*: the delta at which beta* hits zero
oster_delta <- function(rmax) b_til * (r_til - r_dot) / ((b_dot - b_til) *
(rmax - r_til))

## Oster's rule of thumb is Rmax = 1.3 x R^2 of the long regression; show two
harsher choices too
rmax <- c(rmax_1.3 = 1.3 * r_til, rmax_2 = 2 * r_til, rmax_1 = 1)
```

```

rbind(beta_star_delta1 = sapply(rmax, oster_beta, delta = 1),
      delta_star       = sapply(rmax, oster_delta))
#>                rmax_1.3 rmax_2 rmax_1
#> beta_star_delta1    0.5949 0.5142 0.2711
#> delta_star         18.2072 5.4622 1.7568

```

Read the two rows. `beta_star_delta1` is the estimate if unobservables were exactly as important as the observables ($\delta = 1$); the finding is called robust if the interval from the controlled estimate to this number excludes zero. `delta_star` is how many times stronger than the observables the unobservables would have to be to erase the effect; Oster's standard is $\delta^* > 1$. Here even $R_{\max} = 1$ leaves $\delta^* = 1.8$. This is the simplified formula from the paper; the `robomit` package computes the exact one.

8.2 Partial R^2 and the robustness value

```

sens <- sensemakr(model = reg, treatment = "D",
                  benchmark_covariates = "wordsum",
                  kd = 1:3)

sens
#> Sensitivity Analysis to Unobserved Confounding
#>
#> Model Formula: Y ~ D + pareduc + wordsum + female + black + otherace + exper
+
#>      I(exper^2)
#>
#> Null hypothesis: q = 1 and reduce = TRUE
#>
#> Unadjusted Estimates of ' D ':
#>   Coef. estimate: 0.629
#>   Standard Error: 0.032
#>   t-value: 19.98
#>
#> Sensitivity Statistics:
#>   Partial R2 of treatment with outcome: 0.102
#>   Robustness Value, q = 1 : 0.285
#>   Robustness Value, q = 1 alpha = 0.05 : 0.262
#>
#> For more information, check summary.

```

```
plot(sens)
```

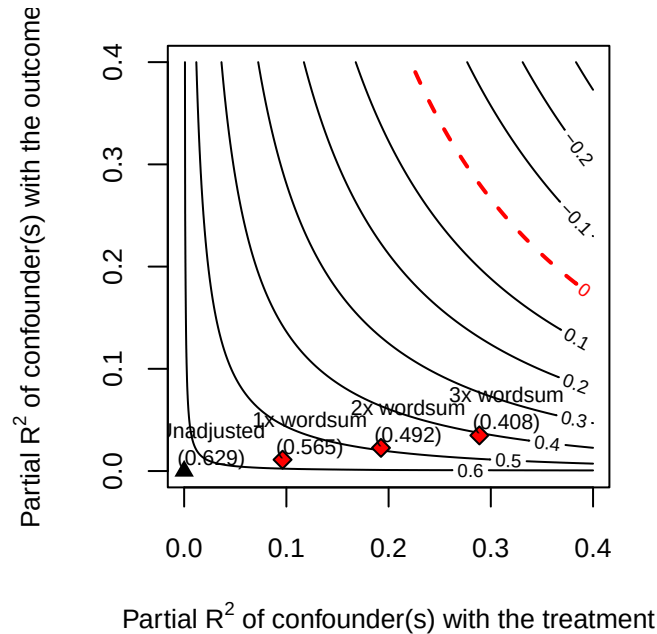


Figure 3: Contours of the adjusted estimate as a function of how much residual variation in treatment and outcome an unobserved confounder explains. Benchmarks show multiples of the vocabulary score.

Read the **robustness value**: it is the share of residual variation in both the treatment and the outcome that a confounder would need to explain to reduce the estimate to zero. Compare it to the benchmark: if a confounder three times as strong as measured verbal ability would be needed, the finding is reasonably robust; if half of one would do it, it is not.

```
## the robustness value from the t statistic alone: RV = (sqrt(f^4 + 4 f^2) - f^2) / 2, f = |t| / sqrt(df)
tt <- coef(summary(reg))["D", "t value"]; df <- reg$df.residual
f <- abs(tt) / sqrt(df)
c(t = tt, df = df, RV_by_hand = (sqrt(f^4 + 4 * f^2) - f^2) / 2,
  RV_sensemkr = sens$sensitivity_stats$rv_q)
#>           t           df  RV_by_hand RV_sensemkr
#>    19.9781    3500.0000     0.2855     0.2855
```

9 Selection on unobservables: the Heckman correction

Everything above conditioned on X . The last section of the lecture asked what can be done when units select on something we did *not* measure. Heckman's answer is to write down the selection rule and the joint distribution of the two errors, and then compute the gap $E[\epsilon_i | D_i = 1] - E[\epsilon_i | D_i = 0]$ instead of assuming it away.

i Note

The GSS offers no credible *excluded variable*, a variable that moves college completion but has no direct effect on earnings. Without one, the lecture showed that the correction rests on the curvature of ϕ/Φ alone. So this section works on simulated data where the true effect is known, exactly as the lecture's table did (figs/w6_heckman.R).

9.1 The world

```
set.seed(723)
n <- 2000
rho <- 0.5; sigma <- 1; tau <- 1
X <- rnorm(n)      ## the observed control, in both equations
Z <- rnorm(n)      ## the excluded variable: shifts selection, not the outcome
u <- rnorm(n)      ## selection-equation error, variance 1
## outcome error correlated rho with u: that correlation IS selection on
unobservables
eps <- sigma * (rho * u + sqrt(1 - rho^2) * rnorm(n))
D <- as.integer(0.5 * X + Z + u > 0)  ## the selection rule
Y <- 1 + 0.5 * X + tau * D + eps      ## the outcome, one constant effect tau
= 1
dat <- data.frame(Y, D, X, Z)
## OLS with X in the regression: biased upward, because X is not the
confounder
coef(lm(Y ~ X + D, data = dat))["D"]
#>      D
#> 1.601
```

9.2 The two steps by hand

```
## step 1: probit of D on the selection regressors, X and the excluded Z
step1 <- glm(D ~ X + Z, family = binomial(link = "probit"), data = dat)
c_hat <- predict(step1, type = "link")      ## Z_i' gamma-hat
## step 2: the correction term, one expression for both groups
dat$lambda <- ifelse(dat$D == 1, dnorm(c_hat) / pnorm(c_hat),
                     -dnorm(c_hat) / (1 - pnorm(c_hat)))
step2 <- lm(Y ~ X + D + lambda, data = dat)
## D recovers tau; the coefficient on lambda estimates rho * sigma = 0.5
coef(summary(step2))[, c("Estimate", "Std. Error", "t value")]
#>           Estimate Std. Error t value
#> (Intercept)  0.9784    0.04249  23.028
#> X           0.5044    0.02346  21.500
```

```
#> D          1.0222    0.07374  13.863
#> lambda      0.5280    0.05354   9.862
```

The t -test on `lambda` is the test of selection on unobservables. But the standard errors printed above are wrong: `lambda` was estimated in step 1, and ϵ_i is heteroskedastic given D_i . Bootstrap both steps together.

```
two_step <- function(d, selection = D ~ X + Z) {
  s1 <- glm(selection, family = binomial(link = "probit"), data = d)
  ch <- predict(s1, type = "link")
  d$lambda <- ifelse(d$D == 1, dnorm(ch) / pnorm(ch), -dnorm(ch) / (1 -
pnorm(ch)))
  coef(lm(Y ~ X + D + lambda, data = d))[c("D", "lambda")]
}
B <- 200
boot <- replicate(B, two_step(dat[sample(n, replace = TRUE), ]))
apply(boot, 1, sd)      ## compare with the step-2 OLS standard errors above
#>      D  lambda
#> 0.07979 0.05520
```

9.3 Both equations by maximum likelihood

The two-step estimator is the Week 3 idea of “estimate, then plug in.” Fitting the probit and the outcome equation jointly by maximum likelihood (FIML) uses the bivariate normal likelihood directly, returns ρ and σ separately, and gives standard errors that need no bootstrap.

```
library(sampleSelection)
## treatReg: selection equation first, then the outcome equation with D in it
fiml <- treatReg(D ~ X + Z, Y ~ X + D, data = dat)
summary(fiml)
#> -----
#> Tobit treatment model (switching regression model)
#> Maximum Likelihood estimation
#> Newton-Raphson maximisation, 3 iterations
#> Return code 1: gradient close to zero (gradtol)
#> Log-Likelihood: -3623
#> 2000 observations: 1007 non-participants (selection 0) and 993 participants
(selection 1)
#>
#> 8 free parameters (df = 1992)
#> Probit selection equation:
#>
#>      Estimate Std. Error t value      Pr(>|t|)
#> (Intercept)  -0.0107    0.0333   -0.32      0.75
```

```

#> X            0.5669      0.0367    15.46 <0.0000000000000002 ***
#> Z            1.0538      0.0444    23.71 <0.0000000000000002 ***
#> Outcome equation:
#>           Estimate Std. Error t value      Pr(>|t|)
#> (Intercept)  0.9868      0.0426    23.1 <0.0000000000000002 ***
#> X            0.5068      0.0246    20.6 <0.0000000000000002 ***
#> D            1.0054      0.0724    13.9 <0.0000000000000002 ***
#> Error terms:
#>           Estimate Std. Error t value      Pr(>|t|)
#> sigma      1.0145      0.0187    54.1 <0.0000000000000002 ***
#> rho        0.5352      0.0419    12.8 <0.0000000000000002 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#> -----

```

The wage version, where Y is observed only for the $D_i = 1$ rows, is `heckit(selection = D ~ X + Z, outcome = Y ~ X, data = ...)` in the same package.

9.4 Without an excluded variable

```

## the selection equation now has only X: lambda is a smooth function of X and
D
s1 <- glm(D ~ X, family = binomial(link = "probit"), data = dat)
ch <- predict(s1, type = "link")
dat$lambda_noZ <- ifelse(dat$D == 1, dnorm(ch) / pnorm(ch), -dnorm(ch) / (1 -
pnorm(ch)))
## how much of the correction term do X and D already explain linearly?
summary(lm(lambda_noZ ~ X + D, data = dat))$r.squared
#> [1] 0.999
## what is left identifies tau: the curvature of phi/Phi, and nothing else
coef(summary(lm(Y ~ X + D + lambda_noZ, data = dat)))[c("D", "lambda_noZ"), ]
#>           Estimate Std. Error t value Pr(>|t|)
#> D            1.54597      1.477  1.04654  0.2954
#> lambda_noZ    0.03416      0.909  0.03758  0.9700

```

Compare the standard error on D with the one from the two-step with Z. The excluded variable must move treatment and have no direct effect on Y . That is next week's instrument.

9.5 When the errors are not normal

```

## the common shock is a centred, unit-variance, right-skewed chi-square(1)
v <- (rchisq(n, 1) - 1) / sqrt(2)

```

```

eps_s <- sigma * (rho * v + sqrt(1 - rho^2) * rnorm(n))
D_s <- as.integer(0.5 * X + Z + v > 0)
Y_s <- 1 + 0.5 * X + tau * D_s + eps_s
dat_s <- data.frame(Y = Y_s, D = D_s, X, Z)
two_step(dat_s)                                ## with the excluded variable
#>      D lambda
#> 0.9689 0.4400
two_step(dat_s, selection = D ~ X)            ## without it
#>      D lambda
#> 0.08352 0.80764

```

With an excluded variable the normal formula for the gap is only slightly off. Without one, the estimator *is* the normal formula, and it can land anywhere.

10 Exercises

1. **Estimand discipline.** Redo the analysis targeting the **ATT** throughout (matching, ATT weights, and an ATT version of AIPW). Do the answers move? Explain why the ATE and ATT differ here in terms of who is on the margin of completing college.
2. **Trimming.** Restrict to $0.1 < \hat{e} < 0.9$ and recompute IPW and AIPW. How much does the estimate move, and how many observations did you drop? Write one sentence stating what population your new estimate describes.
3. **Deliberate misspecification, on simulated data.** Double robustness cannot be “confirmed” against observational data, because the causal truth is unknown. Simulate a world with a known ATE, then fit (a) a correct outcome model with a badly wrong propensity model and (b) the reverse. Confirm that AIPW stays close to the truth in both cases while IPW and g-computation each fail in one. This is double robustness in action.
4. **A bad control.** Add **prestige** (occupational prestige) to the covariate set. It is plausibly a *consequence* of holding a degree. Report how the estimate changes and explain, using the Week 5 vocabulary, why the new number answers a different question.
5. **Balance versus fit.** Compute the AUC of the propensity model. Then add interactions until balance improves. Does the AUC go up or down? Discuss what this means for the “propensity score paradox” from lecture.
6. **Effective sample size.** Plot the IPW weights against $\hat{e}$. Identify the ten most heavily weighted observations. Refit without them, how much does the estimate move relative to its standard error?
7. **An excluded variable for college.** Propose a variable in the GSS, or one you would collect, that shifts completing a bachelor’s degree but has no direct effect on earnings. Write the argument for “no direct effect” in three sentences, then list what would break it.

Explain, using the last section, why the correction is worth little without such a variable.

11 Session information

```
## R version and the packages this lab loaded
c(R = R.version.string, sapply(sessionInfo()$otherPkgs, \(p)
as.character(p$Version)))
```

#>	R	sampleSelection
#>	"R version 4.5.1 (2025-06-13)"	"1.2-14"
#>	maxLik	miscTools
#>	"1.5-2.2"	"0.6-30"
#>	boot	lmtest
#>	"1.3-32"	"0.9-40"
#>	zoo	sandwich
#>	"1.8-14"	"3.1-1"
#>	sensemakr	cobalt
#>	"0.1.6"	"5.0.0"
#>	MatchIt	ggplot2
#>	"4.8.1"	"4.0.0"
#>	dplyr	
#>	"1.1.4"	